

Functional Verification of Complete Sequential Behaviors: A Formal Treatment of Discrepancies between System-Level and RTL Descriptions

Carlos Ivan Castro Marquez, Marius Strum, Wang Jiang Chau

School of Engineering, University of São Paulo

Department of Electronic Systems

{cicastro, strum, jcwang}@lme.usp.br

São Paulo, Brazil

Abstract—Formal techniques allow exhaustive verification on circuit design (at least in theory), but due to actual computational limitations, workarounds must always be adopted to check only a portion of the design at a time. Sequential equivalence checking is an effective approach, but it can only be applied between circuit descriptions where a one-to-one correspondence for states, as well as for memory elements, is expected. This paper presents a formal methodology to verify RTL descriptions through direct comparison with high-level reference models. By doing so, there is no need to specify or analyze formal properties, as the complete behavior is already contained in the reference model. We also consider the natural discrepancies between system level and RTL code, including non-matching interface and memory elements, and state mapping. In this manner, we are able to prove the functional coherence for the overall sequential behavior of the design under verification.

Keywords—Formal RTL validation; reference model; finite state machine with data; process concurrency; state sequence coverage.

I. INTRODUCTION

In functional verification, the correctness of the RTL description (the design under verification, DUV) is checked against the system specification, which can be eventually represented by a functionally equivalent model. Simulation-based techniques make use of either the so-called golden models that describe the expected circuit behavior at a higher level of abstraction or sets of assertions added to the DUV's descriptive code. The main advantage of simulation-based verification is its ability to spot quickly easy-to-find bugs, and, in fact, it is the most widespread technique for commercial designs [1]. Nevertheless, cycle-accurate simulation is still far from being efficient enough to provide a basis for exhaustive verification practices. Different techniques have been proposed to circumvent such fact [2,3], but, although productivity can be improved, simulation-based verification state coverage remains reasonably limited.

Formal methods have always been the alternative to simulation whenever verification exhaustiveness is an issue. Model checking is one of the most widely used formal methods, whereby one checks whether the conditions

expressed by a set of properties are fulfilled at every state, or transition, for a finite state machine (FSM) that represents the DUV. Simplifications are adopted, as in [4,5], in order to manage the complexity in two central points of the model checking approach: to seek a comprehensive set of properties and the time each relevant property, added to match FSM behavior, needs to be checked.

Instead of a set of properties, the formal sequential equivalence checking (SEC) may adopt a reference model, to be compared to the functionalities implemented in the DUV. The greatest part of techniques for SEC has been developed to check consistency of circuit descriptions up to the RTL abstraction level, usually, aimed to verify the consistency of gate/transistor-level netlists [6]. Besides the high associated computational complexity, they do not support checking at system level (ESL).

Some works have contributed to the equivalence checking between ESL and RTL descriptions. Extensions of SEC have been proposed in [7-9], where information and/or code are added to the reference model in order to make it comparable to the RTL representation, which, actually, leads to additional complexity in the equivalence checking process. In [7], the operations are based in a subset of ESL where latency and throughput must be informed to the reference model, while in [8] and [9] the descriptions are transformed in CDFGs and DFGs, respectively.

As a form to reduce equivalence engine effort, similarities between the FSMs of the high level (HL) and RTL models can also be exploited by simplifying the latter; since only the functional aspects must be tested, the DUV is stripped of some RTL-specific constructs, drawing it up to the reference model representation, as proposed in [10-12]. In this way, the number of objects to be compared in the SEC is smaller, making the computation lighter; on the other hand, the behavior of signals specific to the RTL model must be verified at a later step.

In the works above, entire sets of states are computed by the checker instead of acting on a state-by-state basis, as in traditional SEC algorithms, where state table filling or

forward/backward traversal [6] are adopted. In [10], for instance, sequences of events of two FSMs are checked; in order to do so, FIFOs and other high-level constructs are used to wrap the RTL description and, thus, abstract it up to the reference model's level. Therefore, in fact, such approach reduces the potential advantage in performing SEC on descriptions that are more abstract. In [11], state sequences consist of short paths between each two FSM control points, which makes the number of paths very large. The method has been devised for a high-level scheduling tool environment and is not of general application.

A more general approach was proposed in [12], for two FSM descriptions that follow the same algorithm. In this work a FSM with data (FSMD), that represents the RTL description, is reduced through the elimination of inputs and outputs, which are specific to this model, and not present in the reference model. The approach is presented loosely, without a due formalism, making it difficult to perceive its limitations.

In this paper, we propose a formal methodology for verification of RTL code by comparing it with a behavioral high-level description. Following the approach from [12], two sets of sequences of states are obtained from finite state machines with data (FSMD), created from both the DUV, and its correspondent high-level model. We take care of branches, loops, and concurrency. Such theoretical framework establishes some guidelines for the RTL implementation, providing for a secure equivalence checking without false negatives.

The rest of the paper is organized as follows. Section II is used for a recollection of some traditional SEC concepts. Sections III and IV deal, respectively, with some difficulties in checking HL against RTL descriptions, and with the formalism that justify our methodology. In Section V, the methodology and algorithm implementation are described, while in Section VI, an example of application is presented. Finally, Section VII states some brief conclusions and future work.

II. SEQUENTIAL EQUIVALENCE CHECKING

In general, two circuit descriptions can be said to be equivalent if their FSM models are equivalent; actually, this is the most common principle applied to check equivalence, although other forms of representation can also be used. Formally, a FSM is defined as the tuple $\{I, O, S, \delta, \lambda, s_0\}$, where I is the set of inputs, O is the set of outputs, S is the set of states, δ is the state transition relation, λ is the update function, and s_0 is the initial state. From this, the equivalence between two FSM can be defined in terms of equivalent states, as follows [6].

Lemma 1. Given two FSMs, M_1 and M_2 , such that $I_1 = I_2$ and $O_1 = O_2$, two states $s_1 \in S_1$ and $s_2 \in S_2$ are equivalent if, starting from these states, any finite sequence of inputs results in the same sequence of outputs when applied to either M_1 or M_2 .

Lemma 2. Two machines FSM_1 , and FSM_2 , are equivalent if $I_1 = I_2$, $O_1 = O_2$, and their initial states, s_{01} and s_{02} , are equivalent.

As seen, the above is a basic notion of equivalence, valid only when identical input and output behaviors are expected.

Conventionally, tools for SEC divide their functions into sequential state scan and combinational proof tasks. As for the sequential part, several algorithms, such as table-filling and forward/backward state traversal, work to find pairs of states to compare within the machines. Then, engines based on binary decision diagram (BDD), automatic test pattern generation (ATPG) or Boolean satisfiability (SAT), take charge of checking local combinational equivalences [13].

III. CHECKING RTL WITH HIGH-LEVEL MODELS

Checking if M_{RTL} conforms to the behavior described by M_{HL} is a problem that cannot be solved through traditional SEC approaches. In particular, we have spotted three issues that we treat in this paper: inequality of signals and variables, unpairable states, and concurrency. They are explained below.

A. Inequality of signals and variables

Let M_{HL} and M_{RTL} be two FSMs that model the behavior of the high level reference model, and of the DUV, respectively. Most high-level descriptions are merely algorithmic, while, on the contrary, RTL implementations contain details such as clock, synchronism, and scheduling. Following the inequality problem, the following sets are defined:

- I_{HL} , O_{HL} , I_{RTL} , and O_{RTL} are the input and output sets (IOS) of M_{HL} and M_{RTL} , respectively;
- $I_{\cap} = I_{HL}$ and $O_{\cap} = O_{HL}$ are the IOS common to both machines;
- $I'_{RTL} = I_{RTL} - I_{\cap}$ and $O'_{RTL} = O_{RTL} - O_{\cap}$ are the IOS belonging to M_{RTL} exclusively.

Checking if M_{RTL} conforms to the behavior specified by M_{HL} must be done considering the fact that their correspondent alphabets are not congruent with each other. This is illustrated in Fig. 1, where transitions are labeled with update relations as “input/output”; the symbol ‘ $\emptyset$ ’ indicates absence of condition/operation. Also, it can be seen that $I'_{RTL} = \{j_3\}$ and $O'_{RTL} = \{p_3, p_4\}$.

For the sake of simplicity, and when possible, the notation $V = I \cup O$ will be used. Thus, it is possible to refer to the sets V_{HL} , V_{RTL} , $V_{\cap}$, e V'_{RTL} . Also, any sequence of states will be denoted as ‘ k ’, and the set of all k for a machine M , as ‘ K ’.

B. Unpairable states

Without loss of generality, M_{RTL} is expected to have a set of variables greater than or equal to that of M_{HL} . Additionally, S_{RTL} may be greater than or equal to S_{HL} . Such facts may cause that S_{HL} contain one or more states with no mapping to S_{RTL} (i.e., with no equivalent state in S_{RTL}) even if $|S_{HL}| = |S_{RTL}|$. In this way, it is not possible to check if

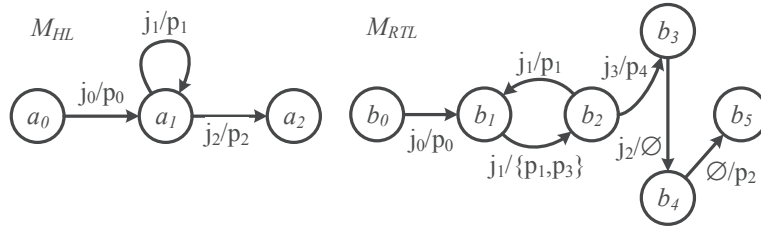


Fig. 1. Behavioral (right) and implementation (left) FSMs with disparities in their alphabets, state mapping, and update functions.

M_{RTL} implements properly the behavior of M_{HL} through a direct state-to-state approach. Such a case is exemplified by states $a_1 \in S_{HL}$ and $b_1, b_2, b_3 \in S_{RTL}$ in Fig. 1. Disregarding the additional output p_3 , b_1 and b_2 would be equivalent with a_1 in extent of the looping transition. Nonetheless, as for the transition leaving to a_2 , no match exists in M_{RTL} , but only a transition leading to b_3 , with an inconclusive input/output relation.

C. Concurrency

Task and operation semantics after RTL scheduling require special treatment when modeled as FSMs, and, in particular, when compared to a high-level model. HL models are supposed to be written, as far as possible, in a hardware-oriented fashion, although languages as, for instance, C/C++, provide no concurrency constructs. Even when deploying system description languages (SDL) such as SystemC, or SpecC, there is no guarantee of one-to-one function or process mapping with respect to the RTL description.

As an example, let us consider the code snippets of a subtraction-based greatest common divisor (GCD) calculator in

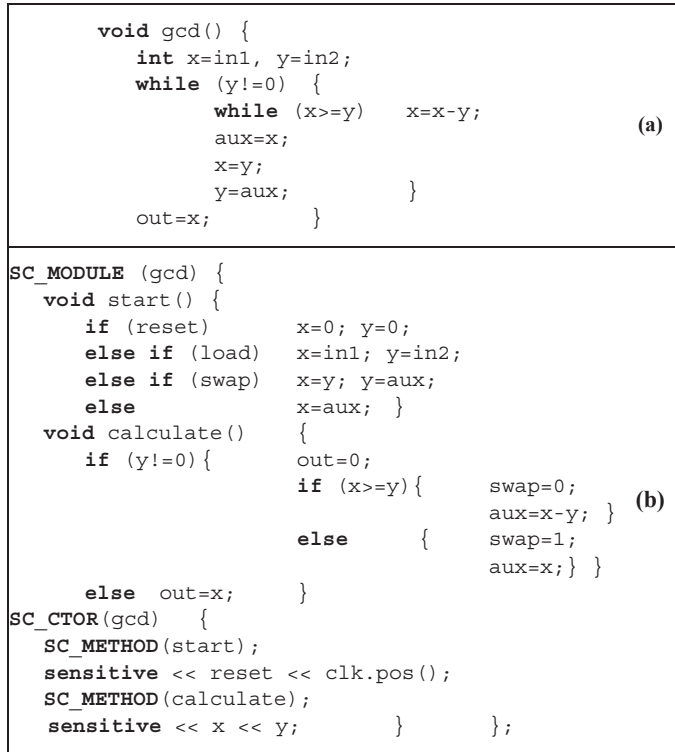


Fig. 2. C function (a) and SystemC module (b) for GCD calculation.

C (Fig. 2a) and its corresponding RTL implementation in SystemC (Fig. 2b), composed by two processes (*start* and *calculate* methods). In order to determine if the RTL code is correct, a complete sequential behavior analysis over both descriptions must be conducted to take account for state interleaving between concurrent state machines.

IV. FORMAL VERIFICATION WITH HIGH-LEVEL MODELS

A. Sequence Reduction

To apply equivalence checking between RTL and HL descriptions, the former must be stripped off from all variable reference specific of such an abstraction level. In our proposal, this is done through the operations defined below.

Definition 1. Given a sequence of states, k_{RTL} , the Reduction function, $R(k_{RTL}) : V_{RTL} \rightarrow V_{\cap}$, is defined as a set of operations that reduce the variables in k_{RTL} only to those belonging to $V_{\cap}$, i.e., the result of $R(k_{RTL})$ is a sequence, k^R_{RTL} , at which $V'_{RTL} = \{\emptyset\}$. The operations performed by R are constant propagation, removal, and grouping, and are explained next.

1) Constant propagation. The values of variables belonging to V_{RTL} , which have data interdependencies with variables belonging to $V_{\cap}$, are propagated to these. The resulting sequence is named k^1_{RTL} .

2) Removal. All inputs and outputs whose variables belong to V'_{RTL} are removed from k^1_{RTL} , and replaced by the symbol ' $\emptyset$ '. The resulting sequence is named k^2_{RTL} .

3) Grouping. Let s_i and $q = \{s_{i+1}, \dots, s_n\}$ be a state and a subsequence of states, respectively, contained in k^2_{RTL} , such that s_i precedes q immediately. Given the conditions: a) the input of s_i , $i_i \in I_{\cap}$, is not unspecified; b) the inputs of the states in q are unspecified, then, s_i and q are grouped, forming a new single state $s_{(i..n)}$, whose output transition is set to be triggered by the input specified in s_i , producing the outputs specified in q , and pointing to the same state, s_{n+1} , pointed by s_n . Idle states, i.e., with no input and output specified, are grouped, indifferently, with the state immediately preceding, or succeeding. The resulting sequence is named k^R_{RTL} .

An example of the reduction operation is shown in Fig. 3. In this case, $V_{\cap} = \{a, b, x, y\}$ and $V'_{RTL} = \{d, e, f, u, v, w\}$. At k^1_{RTL} , all values from variables of V'_{RTL} have been propagated to those of $V_{\cap}$. At k^2_{RTL} , all inputs and outputs whose variables belong to V'_{RTL} have been removed, and, finally, all possible groupings have been performed to obtain k^R_{RTL} . Next, we state the conditions for k_{RTL} , k^1_{RTL} , k^2_{RTL} and k^R_{RTL} to show the same behavior with respect to $V_{\cap}$.

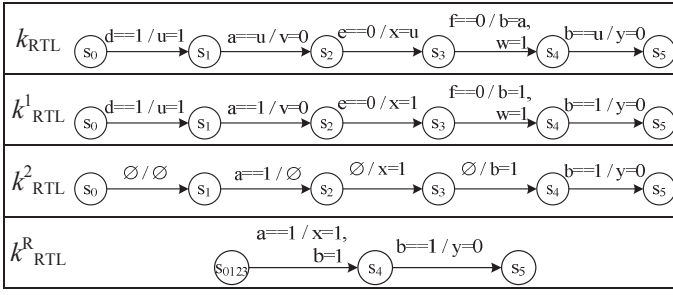


Fig. 3. Example of sequence k_{RTL} reduced by the function R to obtain a sequence k^R_{RTL} .

Postulate 1. Two sequences of states, k_1 and k_2 , have the same behavior with respect to a set of variables, $V = I \cup O$, if the following conditions are satisfied.

- 1) k_1 and k_2 have equal sets of input variables, with the same ordering.
- 2) k_1 and k_2 have equal sets of output variables, and k_2 is a retracted sequence of k_1 .
- 3) If i_a and i_b are two inputs, and o an output, with variables $\{v_{ia}, v_{ib}\} \in I$ and $v_o \in O$, respectively, then i_a triggers the same output assignment, o , at both k_1 and k_2 , be it directly, or indirectly. For the sake of clarity, an output, o , is a direct result of an input, i_a , if it occurs in the same state as i_a , or indirect, if it occurs at a subsequent state, but before any other state where a new input, i_b , appears.

It can be proved that the application of any operation from Definition 1 on sequences of states do not affect the original behavior of the sequence (Postulate 1) with respect to $V_\cap$.

Theorem 1. The propagation (or removal or grouping) operation of the function R does not alter the behavior between k_{RTL} (or k^1_{RTL} or k^2_{RTL}) and k^1_{RTL} (or k^2_{RTL} or k^R_{RTL}) with respect to $V_\cap$.

It is not difficult to demonstrate this theorem for each condition of Postulate 1, but that will be omitted, due to the lack of space in this text. Considering that the property stated in Postulate 1 is transitive, which can be proved quite easily, the following corollary treats the complete reduction operation.

Corollary 1. Any sequence k^R_{RTL} presents the same behavior of a sequence k_{RTL} , with respect to the set of variables $V_\cap$, as long as k^R_{RTL} is obtained by applying the reduction function, R , on k_{RTL} , i.e., $R(k_{RTL}) = k^R_{RTL}$.

B. HL Sequence Expansion

It can be observed that sequences from RTL description, after the application of reduction operation, show the same set of the HL description's symbols, $V_\cap$, allowing them to be compared. Besides that, every RTL reduced state present a valid input condition in every state, therefore, for purposes of easing the comparison of sequences, the same is assumed to the sequences in M_{HL} . In fact, it does not imply any loss of generality, since, in case the assumption is not true, a grouping operation, as stated in Postulate 1, may be applied in order to correct it.

The concepts on equivalence just presented for state sequences can be extended to two machines, since they are composed by sets of sequences. What needs to be guaranteed is that for each sequence in M_{HL} , there is a corresponding extended sequence in M_{RTL} .

The functional equivalence between M_{RTL} and M_{HL} could be determined as long as there is a k^R_{RTL} that describes the same behavior for every k_{HL} , with respect to $V_\cap$. Nevertheless, such perspective can only be considered a “weak” verification because, there is no means to be assured that, in case there is no match between sequences in M_{RTL} and M_{HL} , that means they are not equivalent. Ideally, there should be warranty of absence false negative verification results. In order to avoid the complexity involved in such determinism when any two M_{RTL} and M_{HL} are considered, the present framework includes a “stronger” verification strategy, achieved by restricting the whole set of all possible k_{RTL} . For that, we start by defining such simplified set of sequences as Extended Sequences.

Definition 2. Let $k_{HL} \in K_{HL}$ be a sequence with n states and $k_{RTL} \in K_{RTL}$ another one with m states, such that $m \geq n$. Under the assumption that the following conditions are true, then k_{RTL} is named an extended sequence of k_{HL} .

- 1) Besides the inputs and outputs with variables belonging to V'_{RTL} , k_{RTL} contains, maintaining order and quantity, all the inputs and outputs with variables belonging to $V_\cap$, that are also contained by k_{HL} .

- 2) For each state $s_{HL} \in k_{HL}$, and each input i_{HL} , with $v_{iHL} \in I_\cap$, exists a state $s_{RTL} \in k_{RTL}$, and an input i_{RTL} , with $v_{iRTL} \in I_\cap$, such that $i_{HL} = i_{RTL}$, and the result of $\lambda(s_{HL}, i_{HL})$, when defined, produces the same result as $\lambda(s_{RTL}, i_{RTL})$ directly or indirectly.

An example with three sequences k_{HL} , k_{RTL1} , k_{RTL2} , with $I_\cap = \{v_{i1}, v_{i2}\}$, $O_\cap = \{v_{o1}, v_{o2}\}$, $I'_{RTL} = \{v_{i3}\}$ e $O'_{RTL} = \{v_{o3}, v_{o4}\}$, is shown in Fig. 4. The sequence k_{RTL1} is an extended sequence of k_{HL} , as every output occurs as result of the same input at both sequences. Note that in k_{RTL1} the second instance of o_2 is an indirect result of $\lambda(s_{22}, i_2)$. On the contrary, k_{RTL2} is not an extended sequence of k_{HL} because the second instance of o_2 is a direct result of the input $i_1 \in I_\cap$.

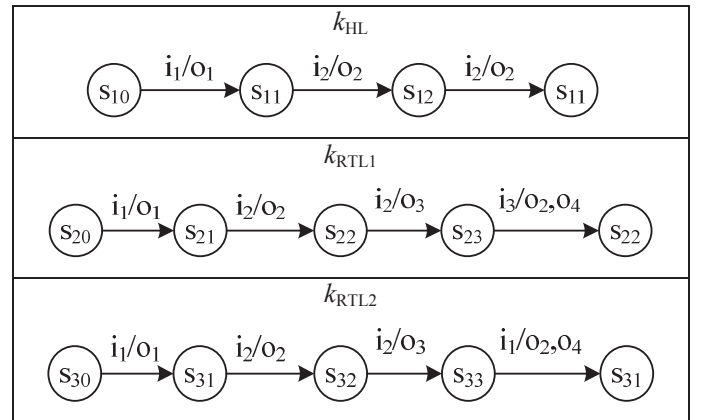


Fig. 4. Example of a sequence k_{RTL1} that is an extended sequence of k_{HL} , and a sequence k_{RTL2} that is not.

It can be proved without difficulty that sequences k_{HL} and k_{RTL} , such that the latter is an extended sequence of the former, have the same behavior with respect to $V_{\cap}$. Also, as direct consequence, due to the transitivity of the property of Postulate 1, k_{HL} and k_{RTL}^R have the same behavior. Therefore, if k_{RTL} is an extended sequence of k_{HL} , we will be able to apply the traditional equivalence checking between k_{HL} and k_{RTL}^R , as defined in section II. Following the rules in Definition 2 may sound restrictive to a RTL description, but, in a top-down design, where a high-level reference model is successively refined, this is a reasonable scenario. Again, the concepts on equivalence just presented for state sequences can be extended to two machines, M_{RTL} and M_{HL} .

V. VALIDATION BY STATE-SEQUENCE EXTRACTION

A. Sequence Extraction

M_{HL} is behaviorally described by a finite set of state sequences, K_{HL} , and that so is M_{RTL} by a set K_{RTL} . Our approach for extracting the set K for a given FSMD obeys the following rules:

- i. Control states generate a new sequence for every decision branch (e.g. k_0 and k_1 in Fig. 5a), unless overridden by rule (iii);
- ii. One sequence may not include more than one state instance, unless it is the last sequence state, unless overridden by rule (iv);
- iii. Rule (i) is overridden if there is a variable whose actual value activates one of the decision branches. In such case, no new sequences are created, and the current sequence continues its extension through the activated branch.
- iv. Rule (ii) is overridden if after overriding rule (i), the variable receives a value not yet used to activate a branch at the control state, and, there is a path from the recurring state to the control state.

These rules are illustrated in Fig. 5. The first two rules are applied to extract the sequences in 5a, where, in absence of the conditions in rule (iii), states s_0 and s_4 generate two sequences each, resulting in a set of four sequences. Also, there is no sequence with more than one state instance, unless it is the last sequence state. In 5b, state s_4 is reached and " $C == 0$ " is true, thus, rule (i) is discarded, the transition to s_1 is taken, and the update " $C = 1$ " is executed. Such setup annuls rule (ii), adding s_1 again to the sequence, and returning to s_4 . The process continues until reaching s_4 for the last time, when no more possible branches are left. In particular, rules (iii) and (iv) have been projected to reduce the effort at the time of comparing K_{HL} and K_{RTL} . Although this was a simple example, it can be seen that by applying such rule, the total number of states to check, for one machine, was reduced in more than 50%.

B. Algorithm for SEC between HL and RTL descriptions

The algorithm that performs SEC between two machines, M_{HL} and M_{RTL} , is presented in Fig. 6. It consists of two main

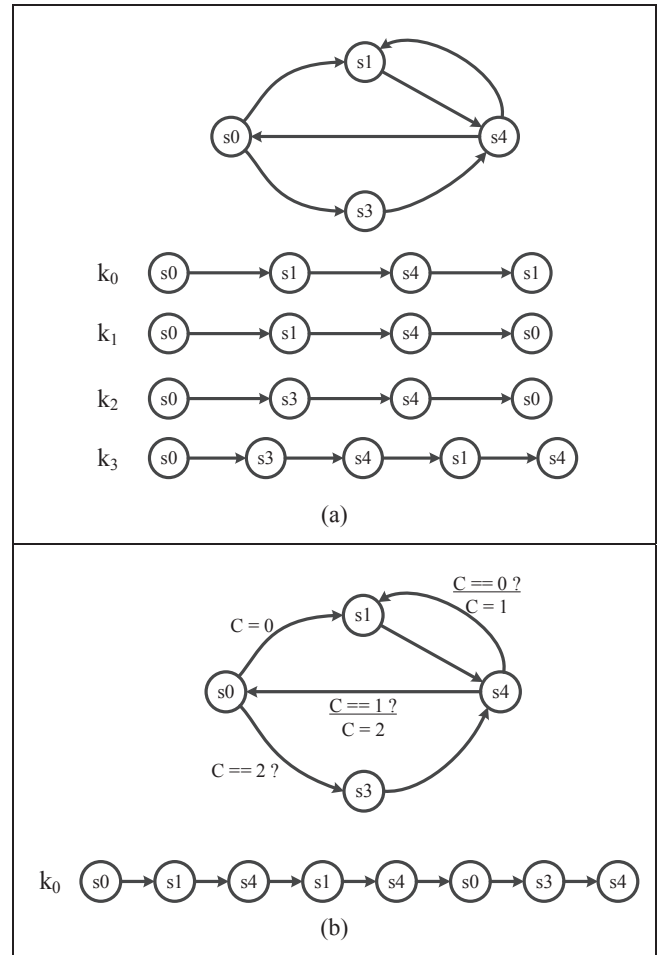


Fig. 5. Sequence extraction from FSMD: (a) simple, (b) loop-control-based.

loops for comparing the two sets of sequences, K_{HL} and K_{RTL}^R . Initially, the algorithm verifies, one by one, if the inputs from each sequence are equivalent. If the end of w_{iHL} is reached without finding a pair for each i_{HL} , then the current k_{RTL}^R is determined as not equivalent to the current k_{HL} ; otherwise, it proceeds to compare w_{oHL} and w_{oRTL}^R . At this step, it is not only checked that every o_{HL} has an o_{RTL} equivalent, but also that they are result of the same input. If, after comparing all the sequences from both K_{HL} and K_{RTL}^R , the two conditions from Lemma 2 are met, then a possible result is obtained.

VI. THE GCD EXAMPLE

The source code excerpts from Fig. 2 are used here to illustrate the application of the definitions above. In Fig. 7, the FSMD models for both C (M_{HL}) and SystemC (M_{RTL0} , M_{RTL1}) descriptions are depicted (number labels are used to differentiate two or more transitions between a single pair of states,). Note that M_{RTL0} is a looping process that is triggered by the clock, starting always at b_{00} . Additionally, M_{RTL1} is always triggered by changes in x or y . In the lower box's left side, the possible state sequences for M_{HL} are shown; if the RTL code is correct, sequences must exist in M_{RTL0} and M_{RTL1} such that all the conditions in Section IV hold. As shown in the figure, it is possible to extract three sequences from the machines that model the SystemC description, which may be

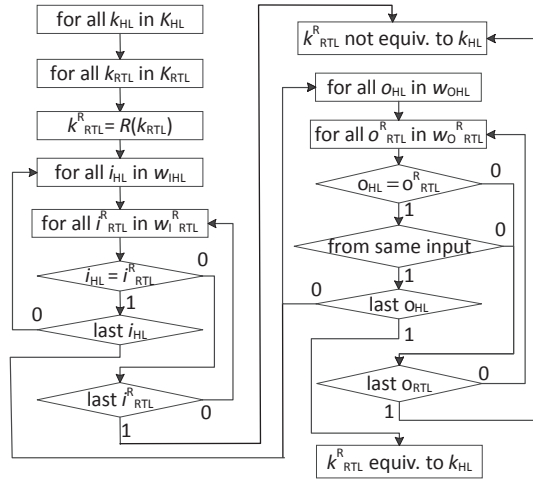


Fig. 6. Algorithm for SEC between two machines M_{HL} and M_{RTL} .

reduced and, in a corresponding manner, compared with those of the high-level version.

VII. CONCLUSIONS AND FUTURE WORK

We presented a theoretical framework to solve the problem of verifying formally a RTL description through direct comparison with its corresponding behavioral model. Given the fact that we work on a FSMD basis, instead of pure FSM, and that we use a SMT solver for word-level checking, we could demonstrate that, through our methodology, it is possible to perform exhaustive verification of hardware descriptions when a high-level reference model is available. This work was particularly adapted to SystemC designs, although the methodology is general enough to apply it to other HDL. As shown in Section III, the approach cares about computation and communication aspects present only in the reference model. We are still working on extending our formalism for the validation of RTL-specific elements, which are usually more synthesis-oriented.

REFERENCES

- [1] Talupur, M., "Hardware model checking: Status, challenges, and opportunities," in *Formal Methods in Computer-Aided Design (FMCAD)*, 2011, p. 154.
- [2] E. Romero, M. Strum, W. Chau, "Manipulation of Training Sets for Improving Data Mining Coverage-Driven Verification". *Journal of Electronic Testing* 29: 223-236, 2013.
- [3] B. Mammo, D. Chatterjee, D. Pidan, A. Nahir, A. Ziv, R. Morad, and V. Bertacco, "Approximating checkers for simulation acceleration," in *Design, Automation & Test in Europe*, pp. 153-158, 2012.
- [4] H.M. Le, D. Grosse, and R. Drechsler, "Towards analyzing functional coverage in SystemC TLM property checking," in *International Conference on High Level Design Validation and Test Workshop*, 2010, pp. 67-74.
- [5] T. K. Nguyen; J. Sun, Y. Liu, and J. S. Dong, "A model checking framework for hierarchical systems," in *International Conference on Automated Software Engineering*, 2011, pp. 633-636.
- [6] P. Molitor, and J. Mohnke, *Equivalence Checking of Digital Circuits: Fundamentals, Principles, Methods*, Kluwer, 2004.
- [7] 18. SLEC Family Product Overview Calypto. <http://calypto.com/products/slec/overview>. Accessed 20 August 2013.
- [8] A. Koelbl, R. Jacoby, H. Jain, and C. Pixley, "Solver technology for system-level to RTL equivalence checking," in *Design, Automation & Test in Europe*, 2009, pp.196-201.
- [9] K. Hao, F. Xie, S. Ray, and J. Yang, "Optimizing equivalence checking for behavioral synthesis," In *Design, Automation and Test in Europe*, 2010, pp. 1500-1505.
- [10] N. Bombieri, F. Fummi, G. Pravadeelli, and J. Marques-Silva, "Towards Equivalence Checking Between TLM and RTL Models," in *International Conference on Formal Methods and Models for Codesign*, pp. 113-122, 2007.
- [11] C. Lee, C. Shih, J. Huang, and J. Jou, "Equivalence checking of scheduling with speculative code transformations in high-level synthesis," in *Asia and*
- [12] C. Marquez, M. Strum, W.J. Chau, "Formal equivalence checking between high-level and RTL hardware designs". in 14th Latin American Test Workshop (LATW), pp. 1-6, 2013.
- [13] R. Drechsler, "Advanced Formal Verification," Kluwer Academic Publishers, Norwell, MA, USA, 2004.

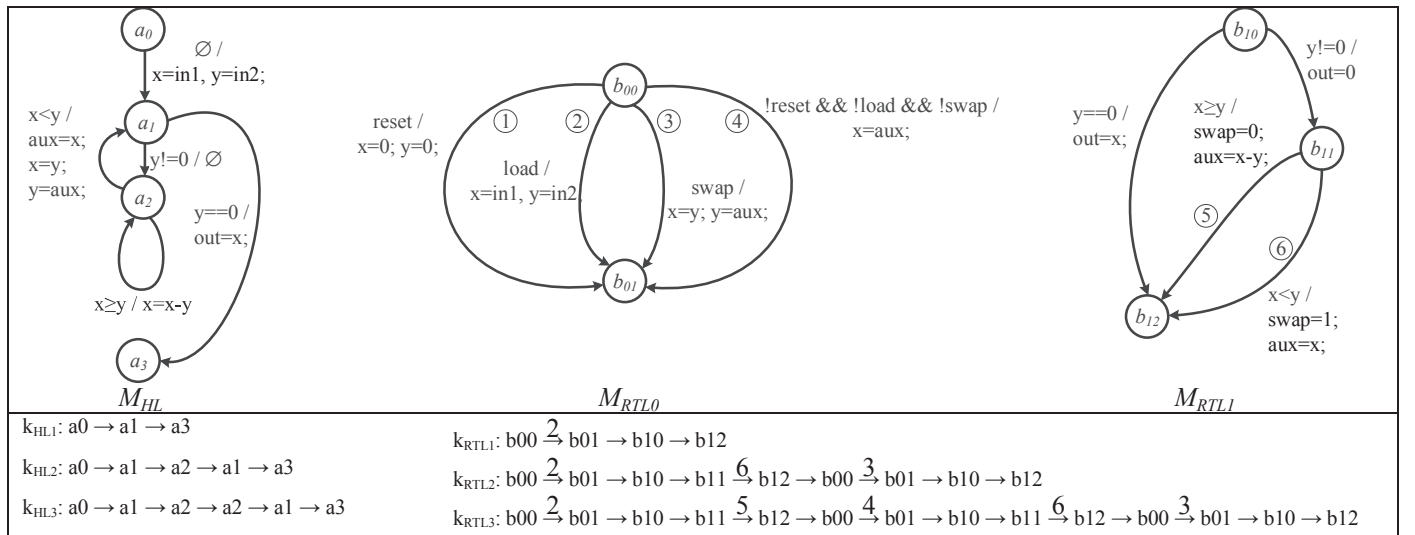


Fig. 7. Example of state sequence equivalence checking for the GCD calculator. FSMD representation (upper box) and state sequences (lower box).